Lecture Notes
Module 0: Preliminaries
From Object-Orientation to Design Patterns

Course: Software Design Patterns
Topic: Structural Relationships, Coupling, & Core Architecture Foundations

Lecture Overview

Before exploring Design Patterns, engineers must master the foundational layout of system objects. Software architecture is not just about writing independent classes; it is about strategically managing the pathways of communication, encapsulation boundaries, and ownership lifecycles between them.

This lecture bridges the gap between high-level conceptual modeling and concrete implementation files. We analyze how classes bind to one another structurally, how these bounds manifest distinctively in Java and C++, and how relationship options dictate system coupling. Mastering these dynamics provides the ultimate prerequisite to understanding why Design Patterns exist.

1. Structural Relationships in OOA, OOD, and OOP

System design progresses through three abstraction paradigms, shifting a relationship from a domain concept down to binary runtime states:

- **OOA (Object-Oriented Analysis):** Focuses on the domain model. We discover real-world conceptual connections without code-level constraints (e.g., 'A Car has an Engine').
- **OOD (Object-Oriented Design):** Focuses on logical architecture. We translate conceptual connections into formal UML specifications, allocating **visibility**, **attributes**, and **explicit types**.
- **OOP (Object-Oriented Programming):** Focuses on **concrete implementation**. We translate UML schemas into native syntax strings, memory footprints, and pointer management policies.

The **five primary structural relationships** follow a spectrum from **weakest coupling** to **strongest coupling**:

- **Dependency (Transient 'Uses-A'):** A short-term **runtime encounter**. Class A utilizes Class B within a single method execution scope. UML Notation: Dashed line with an open arrowhead (- - ->).
- **Association (Structural 'Knows-A'):** A **long-term structural link**. Class A holds an explicit data pathway to reach Class B over time. UML Notation: Solid line with an open arrowhead (--->).

- **Aggregation (Asymmetric 'Has-A'):** A specialized association representing a **part-of assembly** with **weak ownership**. The 'part' object operates **with an independent lifecycle**. UML Notation: Solid line with a hollow diamond (<>---) at the container end.
- **Composition (Strongly-Owned 'Has-A'): A strict lifecycle binding**. The 'part' is **existentially dependent on the 'whole'**. Deleting the container cleanses the parts instantly. UML Notation: Solid line with a filled black diamond (◆---) at the container end.
- **Generalization & Realization ('Is-A' / 'Behaves-As'): Class inheritance (Generalization)** mirrors ancestral state/behavior extraction (———▷). **Interface fulfillment (Realization**) binds a class to a structural API contract (- - ▷).

2. Programmatic Implementations: Java vs. C++

The architectural variance between languages surfaces inside the memory manager. Java abstracts allocations via uniform garbage-collected reference pointers. C++ demands deliberate configuration among stack allocation blocks, raw references, unique pointers, or value structures.

I. Dependency

Achieved by passing the target object as a temporary method argument or locally instantiating it inside an isolated scope framework.

```
// Java Implementation
public class ReportGenerator {
    public void print(Printer printer, String data) {
        printer.output(data); // Alive only within scope
    }
}

// C++ Implementation
class ReportGenerator {
public:
    void print(Printer& printer, const std::string& data) {
        printer.output(data; // Passed by reference, holds no field storage
    }
};
```

II. Aggregation

The container stores a persistent field pointer, but the object instance lands from an external constructor parameter or injection framework.

```
// Java Implementation
public class Department {
    private Professor professor; // Maintained outside this scope
    public Department(Professor professor) {
        this.professor = professor;
    }
}
```

```
// C++ Implementation
#include <memory>
class Department {
private:
    std::shared_ptr<Professor> professor; // Shared resource framework
public:
    Department(std::shared_ptr<Professor> prof) : professor(prof) {}
};
```

III. Composition

The parent instantiates the child internally. Destructors or GC patterns ensure synchronization of lifecycle deaths.

```
// Java Implementation
public class Car {
    private final Engine engine; // Immutable binding parameter
    public Car() {
        this.engine = new Engine(); // Rigid creation pathway
    }
}
```

```
// C++ Implementation
class Car {
private:
    Engine engine; // Managed instantly inside Car's uniform stack frame
public:
    Car() {} // Automatically runs the target Engine initialization allocation
};
```

IV. Generalization (Inheritance)

```
// Java Implementation
public class SavingsAccount extends Account {
    @Override
    public void withdraw() { /* Custom operational execution */ }
}
```

```
// C++ Implementation
class Account {
public:
    virtual void withdraw() = 0; // Pure virtual enforcement
    virtual ~Account() {} // Essential for polymorphic deletion safety
};
class SavingsAccount : public Account {
public:
    void withdraw() override { /* Custom execution */ }
};
```

3. Relationship Strength and System Coupling

Coupling measures the functional dependency density between disparate compilation modules. Strong links manifest severe fragility risks. If Class B scales or adjusts internal layout, Class A yields breaking compile-time compilation fault responses.

Relationship Type	Relative Strength	Coupling Level	Architectural Impact
Dependency	Minimal	Very Weak	High agility. Mechanical changes to B rarely break A signatures.
Association	Low	Weak	Address awareness. Modulating B internal logic shields A structure.
Aggregation	Medium	Moderate	Asymmetric collections. Catalog frameworks retain standalone isolation.
Composition	High	Strong	Forced lifecycles. System A fails compilation if B types go missing.
Generalization	Extreme	Very Strong	White-Box vulnerability. Structural parent changes break subclass assets.

The White-Box Vulnerability of Generalization: Class inheritance fractures baseline structural encapsulation boundaries. Subclasses depend directly on protected structural variables and implementation assumptions of their parent structures. **A modification by a separate engineering unit inside a high-level parent class can echo silent transactional runtime corruption faults across massive downstream codebases**.

4. Architectural Trade-offs: Tight vs. Weak Coupling

Robust engineering rejects flat, universal generalizations like 'weak coupling is universally superior.' Architectural excellence requires balance across **performance profiles**, **system volatility**, and **testing paradigms**.

Tight Coupling Frameworks

- **Performance Optimizations:** Enables aggressive machine compilation inlining routines and direct local memory block layouts (especially within C++ models).
- **Deterministic Compilation:** Static links prevent dynamic tracking lookup traps, keeping processing overhead low during execution paths.
- **Conceptual Constancy:** Enforces strict type safety for domains where relationships are immutable by definition (e.g., a Polygon structure possessing fixed Vertices).
- **The Core Liabilities:** Crystalline fragility, zero runtime flexibility swapping modules, and near-impossible unit-testing boundaries due to an inability to mock out parent resources.

Weak Coupling Frameworks

- **Dynamic Adaptability:** Components connect using polymorphic interface abstractions, enabling hot-swapping behavior layers while systems are live.
- **Asynchronous Team Delivery:** Isolated engineering teams scale separate layers safely once initial interface APIs stand locked and approved.
- **Uncompromised Testability:** Mock configurations seamlessly pass inside constructor entry points, completely isolating logic from database or cloud network parameters.
- **The Core Liabilities:** Elevated architectural complexity, abstract execution chains that are difficult to trace statically, and runtime pointer lookup latency via vtables.

Conclusion: The Gateway to Design Patterns

This establishes the central friction point of scalable programming architecture: **how can engineers assemble enterprise systems without falling victim to structural rigidity**? **Class inheritance offers easy syntax replication but quickly produces rigid, untestable software structures. This observation yields the primary architectural rule**:

> ***"Favor object composition over class inheritance."***

Instead of maintaining towering, nested inheritance columns, modern designs arrange focused components communicating across abstract borders via **weak aggregation** and **realization lines**. This paradigm shift gave birth to Design Patterns. Frameworks such as Strategy, Decorator, State, and Abstract Factory represent pre-packaged configurations of weak coupling designed to shield code systems against volatility. In our next lecture, we unpack the Strategy Pattern to see this architectural trade-off in action.

Seminar Preparation Problems

1. If building a transaction pipeline requiring dynamic rule adjustments based on geo-location parameters at runtime, would you apply Generalization or Composition/Realization? Defend your architectural choice.

2. Draw or list the components of a UML layout capturing the structural nuance between a University possessing Rooms (Composition) versus a University contracting Professors (Aggregation). Indicate diamond variants explicitly.